

Память и указатели

Память

Данные	Адрес
	00000000
	00000001
	00000002
	00000003
	00000004
	00000005
	00000006
	00000007

Память

Данные	Адрес							
	01	00	48	00	90	01	00	00

Память

Данные		Адрес
true	bool	01 00 00 00 00 00 00 00
'H'	char	00 48 00 00 00 00 00 00
400	int	00 90 01 00 00 00 00 00
		00 00 00 00 00 00 00 00

Память

Данные		Адрес
true	bool	01 00 00 00 00 00 00 00
		00 00 00 00 00 01
72	short	48 00 00 00 00 00 02
		00 00 00 00 00 03
400	short	90 01 00 00 00 00 04
		00 00 00 00 00 05
		00 00 00 00 00 06
		00 00 00 00 00 07

Память

Данные		Адрес
Foo	bool	00000000
		00000001
	short	00000002
		00000003
	int	00000004
		00000005
		00000006
		00000007

```
struct Foo {  
    bool a;  
    short b;  
    int c;  
};
```

Память

Данные		Адрес
Foo[2]	Foo	bool 01 00 00000000
		short 48 00 00000001
		int 90 01 00000002
		00 00 00000003
		00 00 00000004
		00 00 00000005
		00 00 00000006
		00 00 00000007
	Foo	00 00 00000008
		7B 00 00000009
		00 00 0000000A
		38 00 0000000B
		FF 00 0000000C
		FF 00 0000000D
		FF 00 0000000E
		FF 00 0000000F

```
struct Foo {  
    bool a;  
    short b;  
    int c;  
};
```

Память

Данные		Адрес
Foo[2]	Foo	bool 01 00 00000000
		short 48 00 00000001
		int 90 01 00000002
		00 00 00000003
		00 00 00000004
		00 00 00000005
		00 00 00000006
		00 00 00000007
		00 00 00000008
		00 00 00000009
		7B 00 0000000A
		38 00 0000000B
		FF 00 0000000C
		FF 00 0000000D
		FF 00 0000000E
		0A 00 0000000F
		00 00 00000010
		00 00 00000011
		00 00 00000012
		00 00 00000013
		00 00 00000014
		00 00 00000015
		00 00 00000016
		00 00 00000017

```
struct Foo {
    bool a;
    short b;
    int c;
};
```

Память

Данные		Адрес
	01 00 48 00 90 01 00 00 00 00	00000000
		00000001
		00000002
		00000003
		00000004
		00000005
		00000006
		00000007
		00000008
		00000009
123	7B 00 38 FF FF FF 0A 00 00 00 00 00 00 00	0000000A
		0000000B
		0000000C
		0000000D
		0000000E
		0000000F
		00000010
		00000011
short*		00000012
		00000013
		00000014
		00000015
		00000016
		00000017
		00000017

Память

Данные	Адрес
	00000000
	00000001
	00000002
	00000003
	00000004
	00000005
	00000006
	00000007
	00000008
	00000009
	0000000A
	0000000B
	0000000C
	0000000D
	0000000E
	0000000F
	00000010
	00000011
	00000012
	00000013
	00000014
	00000015
	00000016
	00000017

Diagram illustrating memory layout and data access:

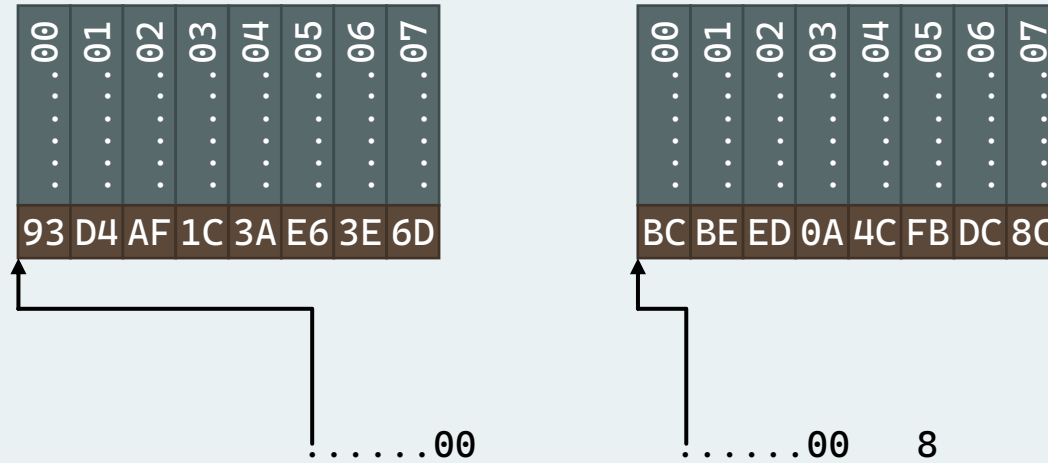
- The memory is organized into rows, each representing a memory address (00000000 to 00000017).
- The data stored at each address is shown in the 'Данные' column.
- The data at address 00000009 is 7B.
- The data at address 0000000A is 00.
- The data at address 0000000B is 00.
- The data at address 0000000C is 38.
- The data at address 0000000D is FF.
- The data at address 0000000E is FF.
- The data at address 0000000F is FF.
- The data at address 00000010 is 0A.
- The data at address 00000011 is 00.
- The data at address 00000012 is 00.
- The data at address 00000013 is 00.
- The data at address 00000014 is 00.
- The data at address 00000015 is 00.
- The data at address 00000016 is 00.
- The data at address 00000017 is 00.

Копирование (memcpy)

...	...	...	...	...	...	...	...
00	01	02	03	04	05	06	07
93	D4	AF	1C	3A	E6	3E	6D

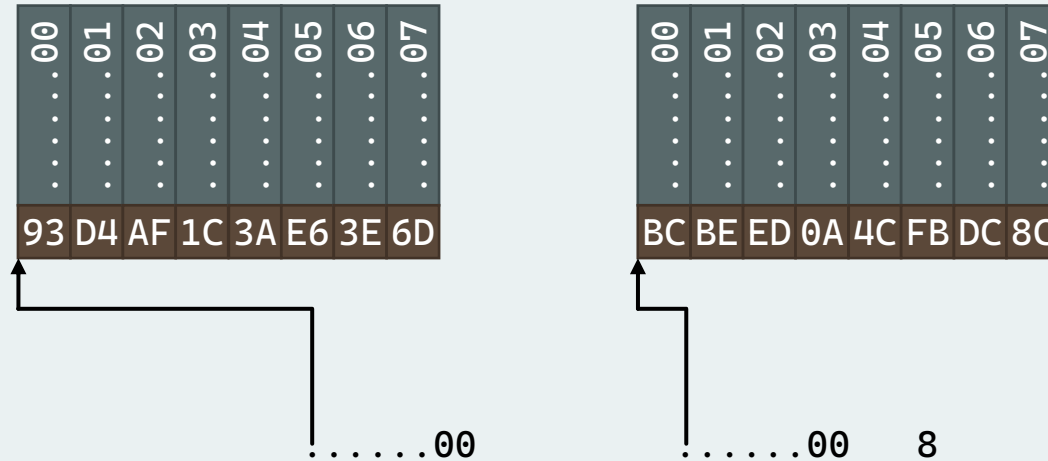
...	...	...	...	...	...	...	...
00	01	02	03	04	05	06	07
BC	BE	ED	0A	4C	FB	DC	8C

Копирование (memcpy)



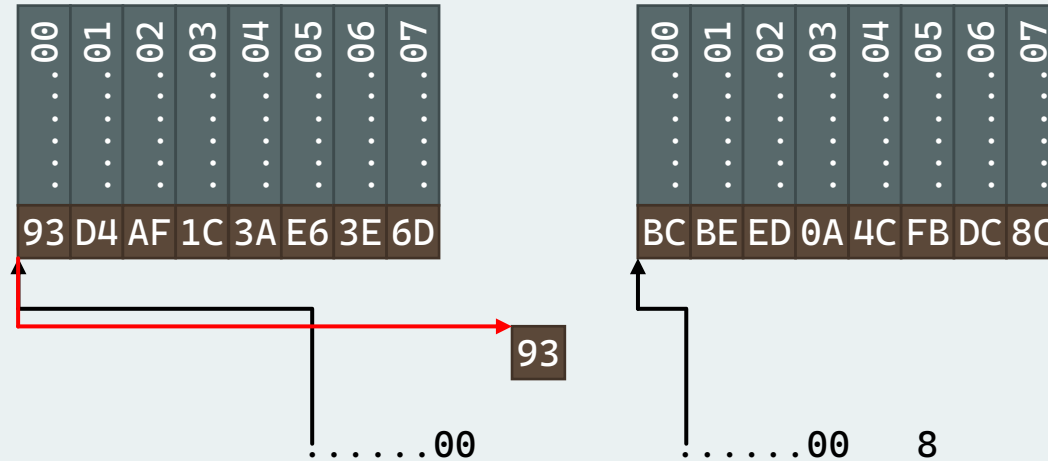
```
void memcpy(const char* src, char* dst, size_t size) {  
  
  
  
  
  
  
}
```

Копирование (memcpy)



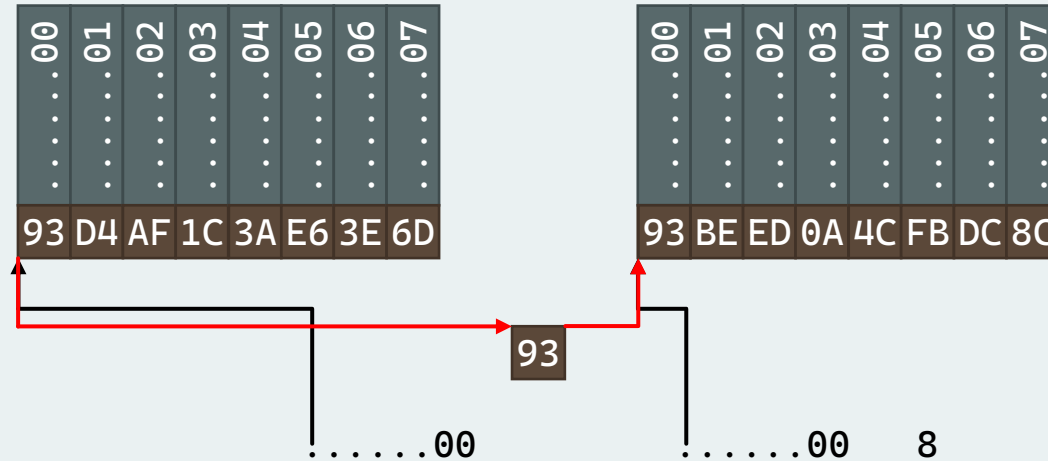
```
void memcpy(const char* src, char* dst, size_t size) {  
    for (size_t i = 0; i < size; ++i) {  
        dst[i] = *(src + i);  
    }  
}
```

Копирование (memcpy)



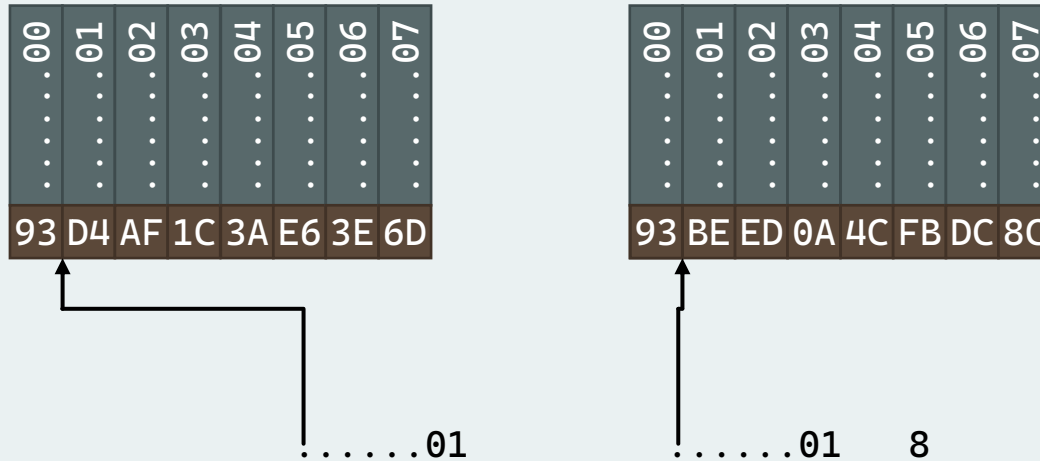
```
void memcpy(const char* src, char* dst, size_t size) {  
    for (size_t i = 0; i < size; ++i) {  
        dst[i] = *(src + i);  
    }  
}
```

Копирование (memcpy)



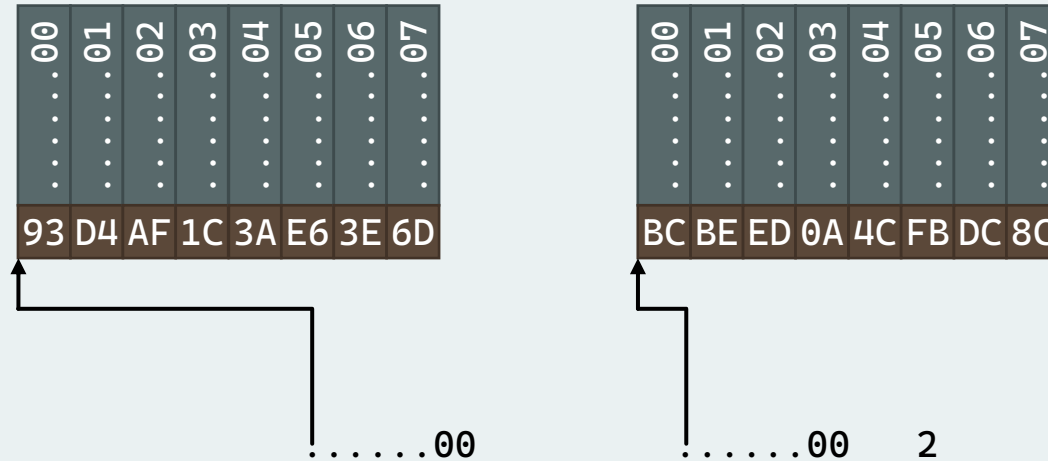
```
void memcpy(const char* src, char* dst, size_t size) {  
    for (size_t i = 0; i < size; ++i) {  
        dst[i] = *(src + i);  
    }  
}
```

Копирование (memcpy)



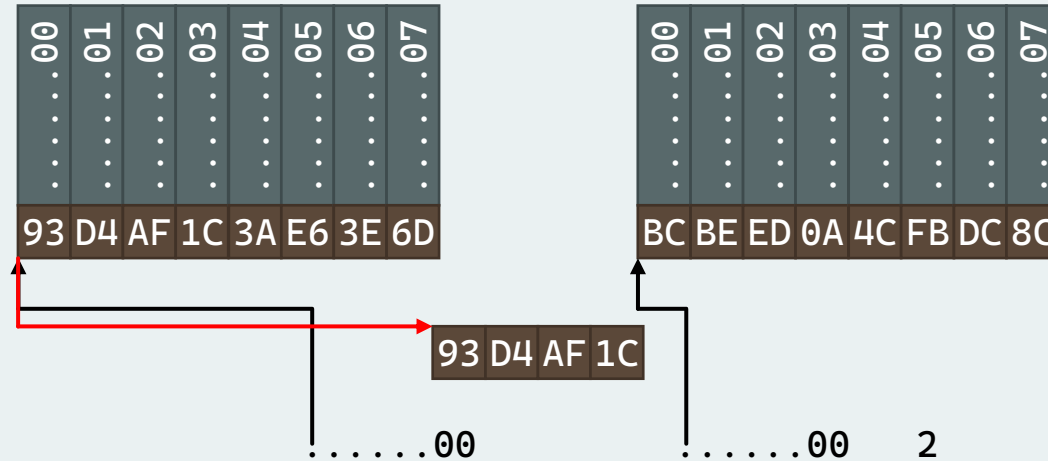
```
void memcpy(const char* src, char* dst, size_t size) {  
    for (size_t i = 0; i < size; ++i) {  
        dst[i] = *(src + i);  
    }  
}
```

Копирование (memcpy)



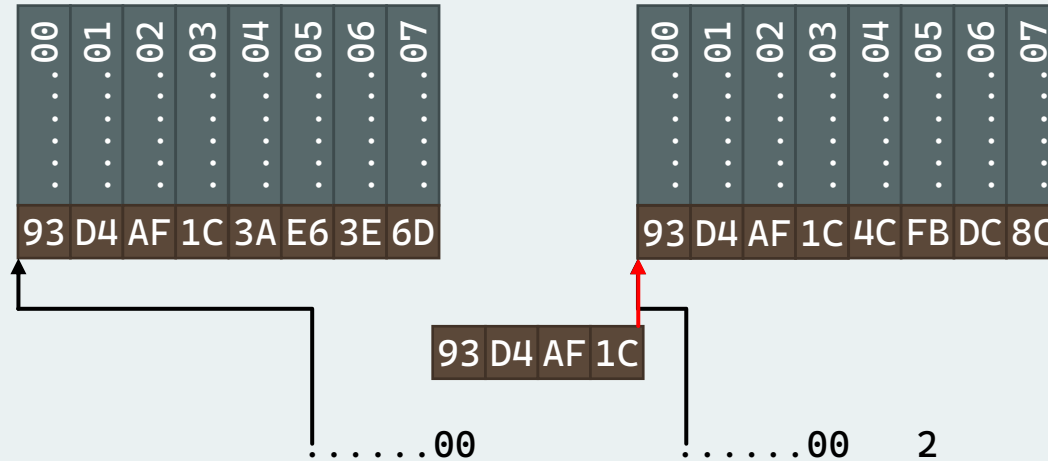
```
void memcpy(const int* src, int* dst, size_t size) {  
    for (size_t i = 0; i < size; ++i) {  
        dst[i] = *(src + i);  
    }  
}
```

Копирование (memcpy)



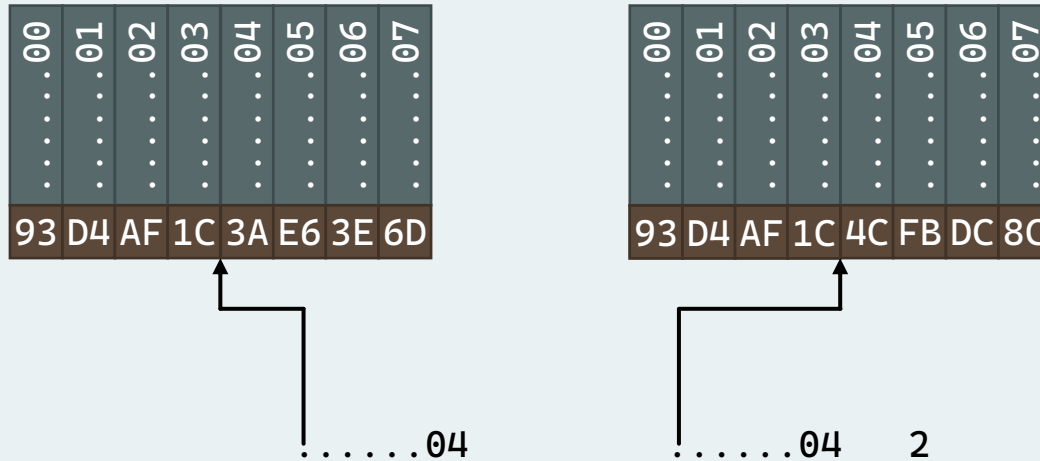
```
void memcpy(const int* src, int* dst, size_t size) {  
    for (size_t i = 0; i < size; ++i) {  
        dst[i] = *(src + i);  
    }  
}
```

Копирование (memcpy)



```
void memcpy(const int* src, int* dst, size_t size) {  
    for (size_t i = 0; i < size; ++i) {  
        dst[i] = *(src + i);  
    }  
}
```

Копирование (memcpy)



```
void memcpy(const int* src, int* dst, size_t size) {  
    for (size_t i = 0; i < size; ++i) {  
        dst[i] = *(src + i);  
    }  
}
```

Копирование (memcpy)

00	01	02	03	04	05	06	07
...	...	...	...	...	...	...	...
93	D4	AF	1C	3A	E6	3E	6D

.....04

00	01	02	03	04	05	06	07
...	...	...	...	...	...	...	...
93	D4	AF	1C	4C	FB	DC	8C

.....04 2

```
void memcpy(const int* src, int* dst, size_t size) {  
    for (size_t i = 0; i < size; ++i) {  
        dst[i] = *(src + i);  
    }  
}
```

Указатель не даёт размер;
Один из способов ограничить
обход - явно задать его

Код

```
int square(int a) {  
    return a * a;  
}
```

Код

```
mov  eax, edi  
imul eax, edi  
ret
```

Код

```
89 F8  
0F AF C7  
C3
```

Код

10000000	10000001	10000002	10000003	10000004	10000005
89	F8	0F	AF	C7	C3

Код

10000000	10000001	10000002	10000003	10000004	10000005
89	F8	0F	AF	C7	C3

```
square(2);
```

Код

10000000	10000001	10000002	10000003	10000004	10000005
89	F8	0F	AF	C7	C3

`call 0x10000000`

константа

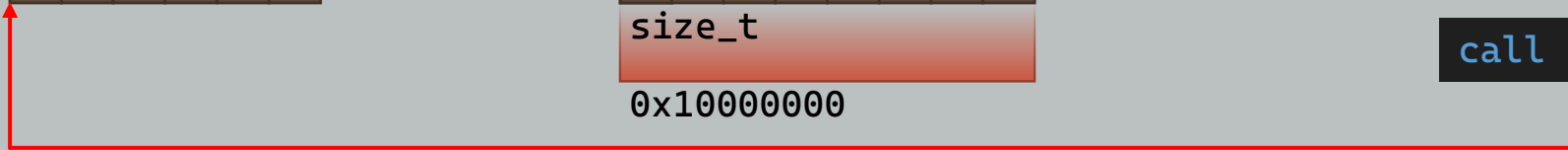


Код

10000000
10000001
10000002
10000003
10000004
10000005
89 F8 0F AF C7 C3

...00
...01
...02
...03
...04
...05
...06
...07
00 00 00 10 00 00 00 00
size_t
0x10000000

`call 0x10000000` константа



Код

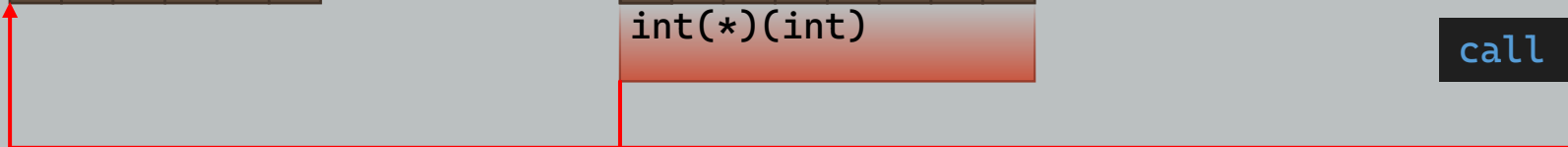
10000000
10000001
10000002
10000003
10000004
10000005
89 F8 0F AF C7 C3

...00
...01
...02
...03
...04
...05
...06
...07
00 00 00 10 00 00 00 00

int(*) (int)

call 0x10000000

константа



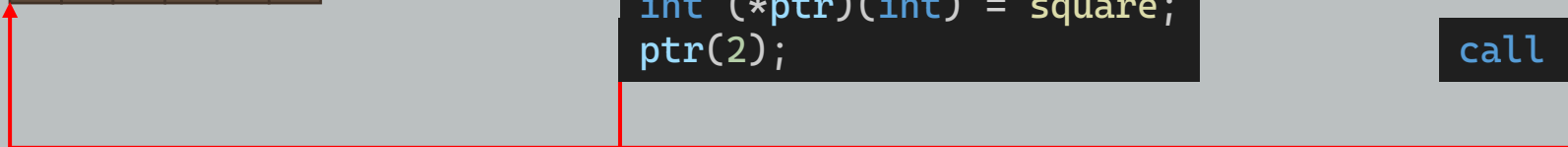
Код

10000000	10000001	10000002	10000003	10000004	10000005
89	F8	0F	AF	C7	C3

```
int (*ptr)(int) = square;  
ptr(2);
```

```
call 0x10000000
```

константа



Код

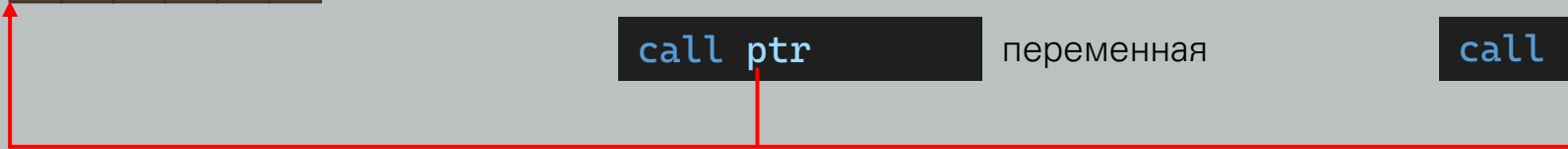
10000000	10000001	10000002	10000003	10000004	10000005
89	F8	0F	AF	C7	C3

`call ptr`

переменная

`call 0x10000000`

константа



transform

```
void transform(int* begin, int* end, int(*map)(int)) {  
  
  
  
  
  
}
```

transform

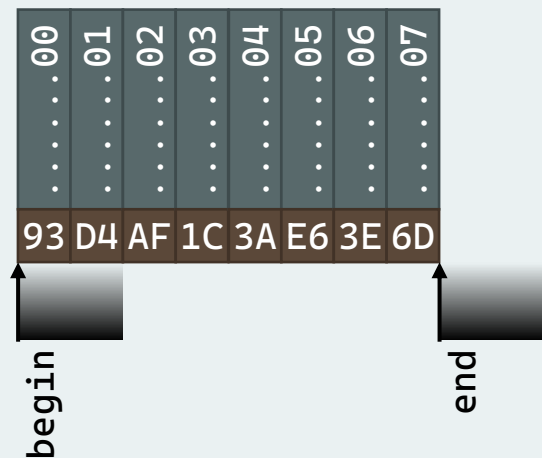
```
void transform(int* begin, int* end, int(*map)(int)) {  
    while (begin != end) {  
        ++begin;  
    }  
}
```

transform

Второй способ явно ограничить обход – передать указатель за конец

```
void transform(int* begin, int* end, int(*map)(int)) {  
    while (begin != end) {  
        ++begin;  
    }  
}
```

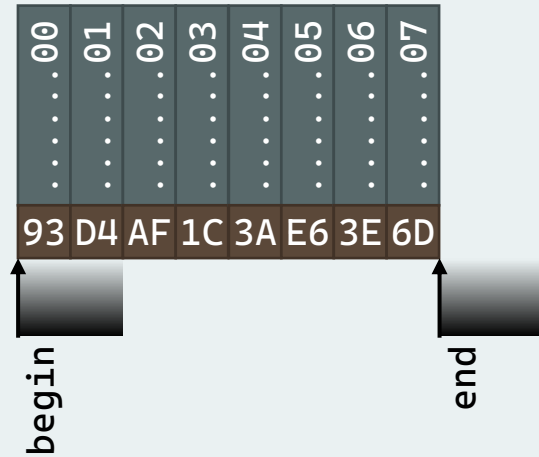
transform



Второй способ явно ограничить обход – передать указатель за конец

```
void transform(int* begin, int* end, int(*map)(int)) {  
    while (begin != end) {  
        ++begin;  
    }  
}
```

transform



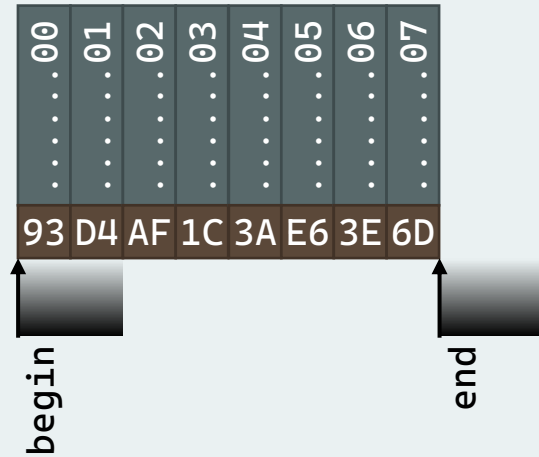
begin+size ↔ begin+end

```
size_t size = end - begin;  
int* end = begin + size;  
int* begin = end - size;
```

Второй способ явно ограничить обход – передать указатель за конец

```
void transform(int* begin, int* end, int(*map)(int)) {  
    while (begin != end) {  
        ++begin;  
    }  
}
```

transform



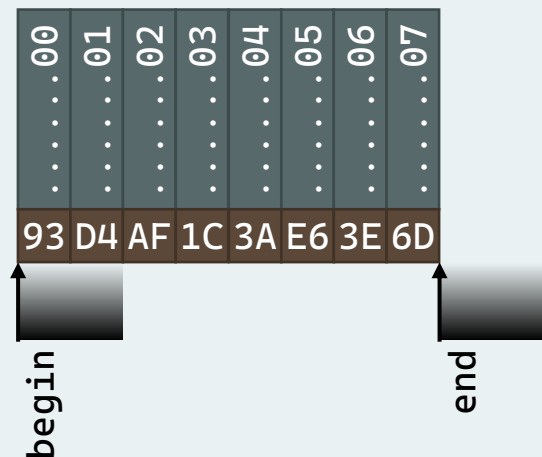
`begin+size` ↔ `begin+end`

```
size_t size = end - begin;  
int* end = begin + size;  
int* begin = end - size;
```

Второй способ явно ограничить обход – передать указатель за конец

```
void transform(int* begin, int* end, int(*map)(int)) {  
    while (begin != end) {  
        *begin = map(*begin);  
        ++begin;  
    }  
}
```

transform



`begin+size` ↔ `begin+end`

```
size_t size = end - begin;  
int* end = begin + size;  
int* begin = end - size;
```

Второй способ явно ограничить обход – передать указатель за конец

```
void transform(int* begin, int* end, int(*map)(int)) {  
    while (begin != end) {  
        *begin = map(*begin);  
        ++begin;  
    }  
}
```

callback («обратный вызов») – вызов функции, переданной заранее, «сзади».

- Параметризация поведения
- Обработка событий (handler)

sort

```
void sort(  
    int* begin,  
    std::size_t count,  
    bool(*cmp)(int, int));
```

sort

Для любого типа?

```
void sort(  
    int* begin,  
    std::size_t count,  
    bool(*cmp)(int, int));
```

sort

Для любого типа?

```
void sort(  
    int* begin,  
    std::size_t count,  
    bool(*cmp)(int, int));
```

```
void anysort(  
    void* begin,  
    std::size_t size,  
    std::size_t count,  
    bool(*cmp)(const void*, const void*));
```

sort

Для любого типа?

```
void sort(  
    int* begin,  
    std::size_t count,  
    bool(*cmp)(int, int));
```

```
void anysort(  
    void* begin,  
    std::size_t size,  
    std::size_t count,  
    bool(*cmp)(const void*, const void*));
```

std::qsort() (C)

sort

Для любого типа?

```
void sort(  
    int* begin,  
    std::size_t count,  
    bool(*cmp)(int, int));
```

```
void anysort(  
    void* begin,  
    std::size_t size,  
    std::size_t count,  
    bool(*cmp)(const void*, const void*));
```

Для логики сортировки
важен только размер типа
и способ его сравнения.

std::qsort() (C)

dispatch

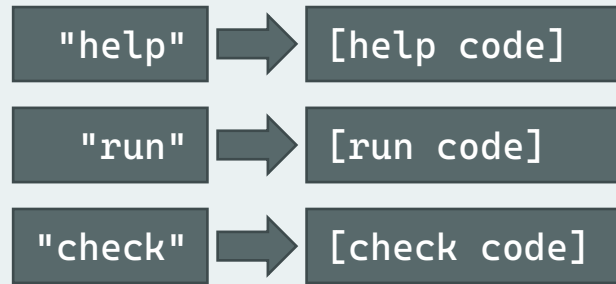
```
if (argv[1] == "help") {  
    // help  
    // ...  
} else if (argv[1] == "run") {  
    // run  
    // ...  
} else {  
    // error  
    // ...  
}
```

dispatch

```
if (argv[1] == "help") {  
    // help  
    // ...  
} else if (argv[1] == "run") {  
    // run  
    // ...  
} else {  
    // error  
    // ...  
}
```



dispatch



dispatch

```
struct Entry {  
    const char* name;  
    int (*action)(int, char**);  
    // ...  
};
```

"help"



[help code]

"run"



[run code]

"check"



[check code]

dispatch

```
struct Entry {  
    const char* name;  
    int (*action)(int, char**);  
    // ...  
};
```

"help"



[help code]

"run"



[run code]

"check"



[check code]

```
const Entry kEntries[] = ...;
```

dispatch

```
struct Entry {  
    const char* name;  
    int (*action)(int, char**);  
    // ...  
};
```

"help"



[help code]

"run"



[run code]

"check"



[check code]

```
const Entry kEntries[] = ...;
```

```
const Entry* find(const Entry* entries, std::size_t count, const char* name);  
find(kEntries, kEntryCount, argv[0])->action(argc, argv); // + проверка
```

dispatch

```
struct Entry {  
    const char* name;  
    int (*action)(int, char**);  
    // ...  
};
```

Например, хранить описание команд в одном месте и доставать их в `--help`.

"help"



[help code]

"run"



[run code]

"check"



[check code]

```
const Entry kEntries[] = ...;
```

```
const Entry* entries, std::size_t count, const char* name);  
  
find(kEntries, kEntryCount, argv[0])->action(argc, argv); // + проверка
```

Спасибо

Алдашкин Кирилл
@canitarr

